

# Παράλληλη Επεξεργασία Διανυσματικών Συζεύξεων με χρήση Locality-Sensitive Hashing στο Apache Spark

## ΠΕΡΙΛΗΨΗ

Η παρούσα εργασία εξετάζει ένα κλασικό πρόβλημα διαχείρισης δεδομένων: την αναζήτηση ζευγαριών από διανύσματα που βρίσκονται πολύ κοντά στον χώρο. Πιο συγκεκριμένα, δεδομένων δύο μεγάλων συνόλων με πολυδιάστατα διανύσματα ( $R$  και  $S$ ), ο στόχος είναι η εύρεση όλων των ζευγαριών που έχουν Ευκλείδεια απόσταση μικρότερη ή ίση με μια τιμή  $\theta$ . Η δυσκολία του προβλήματος έγκειται στο ότι αυτή η τιμή  $\theta$  δεν είναι γνωστή εκ των προτέρων, αλλά ορίζεται δυναμικά από τον χρήστη κατά την εκτέλεση του προγράμματος.

Ο πιο προφανής τρόπος επίλυσης (brute-force) απαιτεί την εξαντλητική σύγκριση κάθε διανύσματος του  $R$  με κάθε διάνυσμα του  $S$ . Ωστόσο, όπως έδειξαν οι αρχικές δοκιμές, αυτή η προσέγγιση είναι εξαιρετικά αργή. Αν εφαρμοζόταν στο μεγάλο σύνολο δεδομένων του 1 εκατομμυρίου διανυσμάτων (SIFT1M), ο χρόνος υπολογισμού θα ήταν πρακτικά απαγορευτικός.

Για την αντιμετώπιση του προβλήματος, υλοποιήθηκε στο Apache Spark ένας προσεγγιστικός αλγόριθμος βασισμένος στην τεχνική Locality-Sensitive Hashing (LSH). Η τεχνική αυτή λειτουργεί τοποθετώντας τα κοντινά διανύσματα σε κοινούς «κουβάδες» (buckets), γλιτώνοντας έτσι το σύστημα από εκατομμύρια άσκοπες συγκρίσεις και εκμεταλλευόμενη πλήρως την παράλληλη επεξεργασία.

Μέσα από εξαντλητικές δοκιμές (Grid Search) σε ένα μικρό δείγμα δεδομένων, διαπιστώθηκε ότι με 3 Hash Tables και Bucket Length 100.0, ο αλγόριθμος εντοπίζει τα σωστά ζευγάρια με ποσοστό επιτυχίας (ανάκληση) 99.81% σε μόλις 1.19 δευτερόλεπτα. Στη συνέχεια, η χρήση αυτών των βέλτιστων παραμέτρων στο μεγάλο σύνολο δεδομένων SIFT1M επιβεβαίωσε ότι η προτεινόμενη λύση είναι ταχύτατη και κλιμακώσιμη (scalable) ακόμα και σε μεγάλους όγκους δεδομένων.

## 1. Εισαγωγή και Ορισμός του Προβλήματος

Η σύγκριση και η εύρεση ομοιοτήτων μεταξύ δεδομένων είναι ένα από τα πιο συνηθισμένα προβλήματα στην

πληροφορική. Στη συγκεκριμένη περίπτωση, το πρόβλημα αφορά τον εντοπισμό "κοντινών" διανυσμάτων ανάμεσα σε δύο μεγάλα σύνολα δεδομένων. Πιο αναλυτικά, δίνονται δύο σύνολα πολυδιάστατων διανυσμάτων, το  $R$  (βάση δεδομένων) και το  $S$  (ερωτήματα). Το ζητούμενο είναι να βρεθούν όλα τα πιθανά ζευγάρια  $(r, s)$ , όπου το ένα διάνυσμα ανήκει στο  $R$  και το άλλο στο  $S$ , των οποίων η Ευκλείδεια απόσταση δεν ξεπερνά μια συγκεκριμένη τιμή  $\theta$ . Μια βασική ιδιαιτερότητα του προβλήματος είναι ότι η τιμή του  $\theta$  δεν είναι σταθερή, αλλά την ορίζει ο χρήστης δυναμικά κάθε φορά που τρέχει το πρόγραμμα.

Ο πιο απλός και προφανής τρόπος για να λυθεί αυτό το πρόβλημα είναι η μέθοδος της εξαντλητικής αναζήτησης (brute-force). Σε αυτή τη μέθοδο, που συνήθως αποτελεί τη βάση σύγκρισης (baseline), το πρόγραμμα παίρνει ένα-ένα τα διανύσματα του συνόλου  $R$  και υπολογίζει την απόστασή τους με όλα τα διανύσματα του συνόλου  $S$  (καρτεσιανό γινόμενο). Αν και αυτή η προσέγγιση εγγυάται ότι θα βρεθεί η 100% ακριβής λύση, έχει τεράστιο υπολογιστικό κόστος. Όταν ο όγκος των δεδομένων μεγαλώνει και φτάνει τα εκατομμύρια εγγραφές, οι μαθηματικοί υπολογισμοί που απαιτούνται αυξάνονται εκθετικά. Το αποτέλεσμα είναι ο χρόνος εκτέλεσης να γίνεται πρακτικά απαγορευτικός.

Για να ξεπεραστεί αυτό το εμπόδιο, η λύση που προτείνεται και υλοποιείται στην παρούσα εργασία δεν αναζητά την απόλυτα ακριβή λύση, αλλά μια προσεγγιστική (approximate). Ο σκοπός είναι να θυσιαστεί ένα πολύ μικρό ποσοστό ακρίβειας προκειμένου ο χρόνος αναζήτησης να μειωθεί δραματικά. Για να επιτευχθεί αυτό, χρησιμοποιείται ο αλγόριθμος Locality-Sensitive Hashing (LSH). Ο αλγόριθμος αυτός λειτουργεί "έξυπνα", αποφεύγοντας τις άσκοπες συγκρίσεις μεταξύ διανυσμάτων που βρίσκονται ξεκάθαρα πολύ μακριά το ένα από το άλλο.

Τέλος, επειδή η διαχείριση τεράστιου όγκου δεδομένων (Big Data) απαιτεί μεγάλη επεξεργαστική ισχύ, η υλοποίηση του συστήματος σχεδιάστηκε εξ ολοκλήρου στο περιβάλλον του Apache Spark. Το Spark επιτρέπει

στον κώδικα να εκτελείται παράλληλα, μοιράζοντας τον φόρτο εργασίας στους διαθέσιμους επεξεργαστές του συστήματος. Έτσι, εξασφαλίζεται ότι η λύση είναι κλιμακώσιμη (scalable) και μπορεί να διαχειριστεί μεγάλες βάσεις δεδομένων, όπως το SIFT1M, αποδοτικά και γρήγορα.

## 2. Προτεινόμενη Αλγοριθμική Λύση

### 2.1 Ο Αλγόριθμος Locality-Sensitive Hashing (LSH)

Για την αντιμετώπιση του απαγορευτικού υπολογιστικού κόστους της εξαντλητικής αναζήτησης, η παρούσα εργασία προτείνει τη χρήση του αλγορίθμου Locality-Sensitive Hashing (LSH), και συγκεκριμένα της παραλλαγής Bucketed Random Projection. Ο LSH ανήκει στην κατηγορία των προσεγγιστικών αλγορίθμων και σχεδιάστηκε ειδικά για την ταχεία εύρεση κοντινών γειτόνων (Nearest Neighbors) σε πολυδιάστατους χώρους.

Η βασική ιδέα πίσω από τον LSH είναι σχετικά απλή αλλά εξαιρετικά αποδοτική: αντί να συγκρίνεται κάθε διάνυσμα με όλα τα υπόλοιπα, ο αλγόριθμος χρησιμοποιεί ειδικές συναρτήσεις κατακερματισμού (hash functions). Αυτές οι συναρτήσεις προβάλλουν τα πολυδιάστατα δεδομένα σε έναν χώρο χαμηλότερων διαστάσεων και τα τοποθετούν σε συγκεκριμένους «κάδους» (buckets). Το χαρακτηριστικό που κάνει τον LSH να ξεχωρίζει από τις κλασικές συναρτήσεις κατακερματισμού είναι ότι μεγιστοποιεί την πιθανότητα δύο διανύσματα που βρίσκονται γεωμετρικά κοντά το ένα στο άλλο, να καταλήξουν στον ίδιο κάδο (collision).

Αντίθετα, διανύσματα που απέχουν πολύ μεταξύ τους, έχουν ελάχιστη πιθανότητα να βρεθούν στον ίδιο κάδο. Έτσι, κατά τη φάση της σύζευξης (join), το σύστημα συγκρίνει μεταξύ τους μόνο τα διανύσματα που έχουν πέσει στον ίδιο κάδο, αγνοώντας πλήρως τα υπόλοιπα εκατομμύρια διανύσματα της βάσης. Αυτό το "φιλτράρισμα" είναι που μειώνει δραματικά τον απαιτούμενο χρόνο εκτέλεσης.

### 2.2 Καθορισμός Παραμέτρων και Trade-offs

Η συμπεριφορά και η αποδοτικότητα του αλγορίθμου LSH εξαρτώνται άμεσα από τη σωστή ρύθμιση δύο βασικών παραμέτρων (υπερπαραμέτροι). Η κατανόησή τους είναι κρίσιμη, καθώς καθορίζουν τη χρυσή τομή (trade-off) μεταξύ της ταχύτητας του συστήματος και της ακρίβειας των αποτελεσμάτων (ανάκληση).

1. Μήκος Κάδου (Bucket Length): Η παράμετρος αυτή ελέγχει πρακτικά το μέγεθος (ή "πλάτος")

κάθε κάδου στον οποίο καταλήγουν τα διανύσματα. Ένα μεγάλο μήκος κάδου σημαίνει ότι τα "δίχτυα" του αλγορίθμου είναι πιο μεγάλα, άρα περισσότερα διανύσματα θα καταλήξουν στον ίδιο κάδο. Αυτό αυξάνει την πιθανότητα να βρεθούν όλα τα σωστά ζευγάρια (υψηλότερο recall), αλλά ταυτόχρονα αυξάνει και τον αριθμό των υποψήφίων ζευγαριών που πρέπει να συγκριθούν, μεγαλώνοντας έτσι τον χρόνο εκτέλεσης.

2. Αριθμός Πινάκων Κατακερματισμού (Num Hash Tables): Η παράμετρος αυτή καθορίζει πόσες φορές θα επαναληφθεί η διαδικασία του hashing με διαφορετικές, ανεξάρτητες συναρτήσεις. Η χρήση πολλαπλών πινάκων κατακερματισμού δίνει σε δύο κοντινά διανύσματα "δεύτερες ευκαιρίες" να βρεθούν μαζί σε κάποιον κάδο, σε περίπτωση που η πρώτη συνάρτηση τα είχε χωρίσει. Η αύξηση αυτού του αριθμού βελτιώνει αισθητά το ποσοστό επιτυχίας, ωστόσο επιβαρύνει την κατανάλωση μνήμης και τον χρόνο που απαιτείται για την κατασκευή των πινάκων.

## 3. Υλοποίηση και Αρχιτεκτονική

Η ανάπτυξη της λύσης σχεδιάστηκε με γνώμονα τη διαχείριση Μεγάλων Δεδομένων. Για τον λόγο αυτό, δεν γράφτηκε ένα απλό σειριακό πρόγραμμα, αλλά αξιοποιήθηκε το οικοσύστημα του Apache Spark.

### 3.1 Επίτευξη Παραλληλίας και Κλιμακωσιμότητας

Η βασική απαίτηση της εργασίας ήταν η παράλληλη εκτέλεση του αλγορίθμου, ώστε η λύση να είναι scalable (κλιμακώσιμη – δηλαδή να μπορεί να διαχειριστεί ομαλά την αύξηση των δεδομένων προσθέτοντας απλώς περισσότερους επεξεργαστικούς πόρους). Αυτό επιτεύχθηκε με δύο στοχευμένες τεχνικές παρεμβάσεις κατά τη ρύθμιση του συστήματος:

1. Διαχείριση Πυρήνων (Cores): Κατά την αρχικοποίηση του Spark Session (η κεντρική «γέφυρα» επικοινωνίας μεταξύ του κώδικα Python και του πυρήνα του Spark), χρησιμοποιήθηκε η παράμετρος `master("local[*]")`. Ο αστερίσκος (\*) αποτελεί μια εντολή προς το σύστημα να ανιχνεύσει και να δεσμεύσει όλους τους διαθέσιμους πυρήνες (cores) του επεξεργαστή στο περιβάλλον

εκτέλεσης (Google Colab). Με αυτόν τον τρόπο, δημιουργούνται πολλαπλά worker threads, τα οποία εκτελούν τους υπολογισμούς ταυτόχρονα και όχι το ένα μετά το άλλο.

2. Κατάτμηση Δεδομένων (Partitioning): Κατά τη φόρτωση του τεράστιου συνόλου δεδομένων SIFT1M, εφαρμόστηκε ρητή κατάτμηση (parallelize) σε 100 partitions. Η πρακτική αυτή είναι ζωτικής σημασίας: το Spark παίρνει αυτά τα 100 κομμάτια και τα μοιράζει στους διαθέσιμους πυρήνες. Έτσι, κάθε πυρήνας επεξεργάζεται ένα διαφορετικό partition ταυτόχρονα, μειώνοντας δραματικά τον συνολικό χρόνο.

### 3.2 Δομές Δεδομένων και Διαχείριση Μνήμης

Για την αναπαράσταση των δεδομένων χρησιμοποιήθηκαν τα Spark DataFrames.

Επιπλέον, επειδή ο αλγόριθμος χρειάζεται να διαβάσει τα δεδομένα πολλές φορές, εφαρμόστηκαν δύο τεχνικές βελτιστοποίησης:

- Μορφή Parquet: Τα αρχικά δεδομένα μετατράπηκαν σε αρχεία Parquet (μια προηγμένη μορφή αποθήκευσης που οργανώνει τα δεδομένα ανά στήλη αντί για ανά γραμμή, προσφέροντας υψηλή συμπίεση και πολύ ταχύτερη ανάγνωση).
- Κλείδωμα στη Μνήμη (Caching): Χρησιμοποιήθηκε η εντολή `cache()` στα DataFrames πριν ξεκινήσουν οι υπολογισμοί. Αυτό αναγκάζει το σύστημα να κρατήσει τα δεδομένα «ζωντανά» στην RAM (Κύρια Μνήμη), αποτρέποντας τις χρονοβόρες και επαναλαμβανόμενες αναγνώσεις από τον σκληρό δίσκο.

### 3.3 Χρήση Εξωτερικών Βιβλιοθηκών

Σύμφωνα με τους αυστηρούς περιορισμούς της εκφώνησης, ο πυρήνας της αλγοριθμικής λύσης (LSH) υλοποιήθηκε αποκλειστικά με τις εγγενείς βιβλιοθήκες του Spark (συγκεκριμένα μέσω της βιβλιοθήκης Spark MLlib που ειδικεύεται στη Μηχανική Μάθηση). Δεν χρησιμοποιήθηκαν έτοιμες εξωτερικές λύσεις τρίτων για τις κατανεμημένες λειτουργίες.

Οι μοναδικές εξωτερικές βιβλιοθήκες της Python που χρησιμοποιήθηκαν, είχαν αυστηρά βοηθητικό ρόλο:

- Βιβλιοθήκη NumPy: Χρησιμοποιήθηκε αποκλειστικά στο πλαίσιο ενός UDF - User Defined Function. Η NumPy αναμενόμενα προσφέρει ταχύτατους μαθηματικούς υπολογισμούς μέσω διανυσματοποίησης, και επιστρατεύτηκε μόνο για να υπολογίσει την ακριβή Ευκλείδεια απόσταση κατά τη φάση υπολογισμού του Ground Truth (της Απόλυτης Αλήθειας, δηλαδή των πραγματικών ζευγαριών με τη μέθοδο της εξαντλητικής αναζήτησης).
- Βιβλιοθήκη Pandas: Αξιοποιήθηκε αυστηρά και μόνο στο τελικό στάδιο του Grid Search (εξαντλητική δοκιμή υπερπαραμέτρων) για την τακτοποίηση και την αισθητικά σωστή οπτικοποίηση των πινάκων με τα αποτελέσματα, χωρίς να επεμβαίνει καθόλου στους υπολογισμούς.

## 4. Πειραματική Αξιολόγηση

Στο παρόν κεφάλαιο παρουσιάζονται τα αποτελέσματα των πειραμάτων που διεξήχθησαν για την αξιολόγηση της ταχύτητας, της ακρίβειας και της κλιμακωσιμότητας του συστήματος.

### 4.1 Περιβάλλον Εκτέλεσης και Δεδομένα

Όλα τα πειράματα πραγματοποιήθηκαν στο δωρεάν υπολογιστικό νέφος του Google Colab, αξιοποιώντας 2 εικονικούς πυρήνες (vCPUs) και περίπου 12GB μνήμης RAM. Για τις ανάγκες της αξιολόγησης χρησιμοποιήθηκε το γνωστό σύνολο δεδομένων SIFT1M. Το σύνολο αυτό αποτελείται από 1.000.000 διανύσματα βάσης (base vectors) και επιπλέον ερωτήματα (queries), όλα σε χώρο 128 διαστάσεων. Για τη φάση της αρχικής δοκιμής και εύρεσης παραμέτρων χρησιμοποιήθηκε ένα πολύ μικρότερο υποσύνολο (siftsmall). Το κατώφλι της Ευκλείδειας απόστασης (παραμέτρος  $\theta$ ) ορίστηκε δυναμικά κατά την εκτέλεση στην τιμή 250.0.

### 4.2 Αξιολόγηση Βασικής Μεθόδου (Baseline)

Για να υπάρχει ένα σαφές μέτρο σύγκρισης, το πρώτο πείραμα αφορούσε την εκτέλεση του αλγορίθμου εξαντλητικής αναζήτησης (cross join) στο μικρό σύνολο δεδομένων (siftsmall). Το δείγμα αυτό αποτελείται από 10.000 διανύσματα βάσης και 100 ερωτήματα (queries). Η μέθοδος αυτή υπολόγισε την ακριβή απόσταση και για τα 1.000.000 πιθανά ζευγάρια ( $10.000 * 100$ ) προκειμένου να βρει την απόλυτη αλήθεια (Ground Truth).

Υπολογισμός Ground Truth (Cross Join)  
Ολοκληρώθηκε σε: 70.01 δευτερόλεπτα  
Για  $\theta=250.0$ , βρέθηκαν 1613 ζευγάρια.

| base_id | query_id | euclidean_dist |
|---------|----------|----------------|
| 31      | 22       | 231.64629      |
| 61      | 11       | 241.64644      |
| 105     | 22       | 208.31706      |
| 107     | 19       | 236.90082      |
| 117     | 22       | 249.948        |

Τα αποτελέσματα έδειξαν ότι για κατώφλι  $\theta = 250.0$  υπάρχουν ακριβώς 1.613 πραγματικά (σωστά) ζευγάρια, αλλά το σύστημα χρειάστηκε 70.01 δευτερόλεπτα για να ολοκληρώσει τους υπολογισμούς σε αυτό το πολύ μικρό δείγμα. Καθίσταται προφανές ότι η εφαρμογή αυτής της μεθόδου στην πλήρη βάση του 1.000.000 διανυσμάτων (όπου θα απαιτούνταν 10 δισεκατομμύρια συγκρίσεις) θα απαιτούσε εξωπραγματικό χρόνο και θα οδηγούσε σε εξάντληση της μνήμης (Out Of Memory), επιβεβαιώνοντας απόλυτα την ανάγκη για τη χρήση του προσεγγιστικού αλγορίθμου LSH.

#### 4.3 Βελτιστοποίηση Παραμέτρων (Grid Search)

Για την εύρεση των ιδανικών ρυθμίσεων του αλγορίθμου LSH, πραγματοποιήθηκε ένα Grid Search. Στη συγκεκριμένη περίπτωση, δοκιμάστηκαν 15 διαφορετικοί συνδυασμοί των παραμέτρων Hash Tables και Bucket Length στο μικρό σύνολο δεδομένων (siftsmall). Για κάθε δοκιμή, καταγράφηκε ο απαιτούμενος χρόνος εκτέλεσης και το Recall (Ανάκληση – το ποσοστό των σωστών ζευγαριών που κατάφερε να βρει ο προσεγγιστικός αλγόριθμος σε σύγκριση με τα 1.613 πραγματικά ζευγάρια του Ground Truth).

HashTables/BucketLength/Χρόνος(sec)/  
ΣωστάΖευγάρια/Recall(%)

|   |       |      |      |        |
|---|-------|------|------|--------|
| 1 | 100.0 | 3.98 | 1550 | 96.09% |
| 1 | 200.0 | 1.43 | 1550 | 96.09% |
| 1 | 300.0 | 1.01 | 1550 | 96.09% |
| 1 | 400.0 | 1.03 | 1550 | 96.09% |
| 1 | 500.0 | 0.96 | 1550 | 96.09% |
| 2 | 100.0 | 1.05 | 1606 | 99.57% |
| 2 | 200.0 | 0.97 | 1606 | 99.57% |
| 2 | 300.0 | 0.95 | 1606 | 99.57% |
| 2 | 400.0 | 0.94 | 1606 | 99.57% |
| 2 | 500.0 | 1.16 | 1606 | 99.57% |

|   |       |      |      |        |
|---|-------|------|------|--------|
| 3 | 100.0 | 1.19 | 1610 | 99.81% |
| 3 | 200.0 | 1.34 | 1610 | 99.81% |
| 3 | 300.0 | 1.33 | 1610 | 99.81% |
| 3 | 400.0 | 1.19 | 1610 | 99.81% |
| 3 | 500.0 | 1.29 | 1610 | 99.81% |

Παρατήρηση επί των αποτελεσμάτων: Διαπιστώνεται ότι η ακρίβεια (Recall) βελτιώνεται αποκλειστικά με την αύξηση των Hash Tables (από 96.09% σε 99.81%), ενώ παραμένει σταθερή όταν αυξάνεται το Bucket Length. Αυτό εξηγείται πλήρως από τη θεωρία του LSH: η προσθήκη νέων Hash Tables λειτουργεί ως "δεύτερη ευκαιρία" για να εντοπιστούν τα κοντινά διανύσματα που ίσως χάθηκαν κατά την πρώτη κατανομή στους κάδους. Αντίθετα, η αρχική τιμή 100.0 για το Bucket Length δημιουργεί ήδη αρκετά "ευρείς" κάδους για το συγκεκριμένο πρόβλημα, οπότε η περαιτέρω αύξησή του (π.χ. σε 300 ή 500) δεν αποκαλύπτει νέα σωστά ζευγάρια, παρά μόνο καθυστερεί ελαφρώς το σύστημα προσθέτοντας άσκοπες συγκρίσεις.

Από την ανάλυση των αποτελεσμάτων, η βέλτιστη ισορροπία (trade-off) μεταξύ ταχύτητας και ακρίβειας προέκυψε επιλέγοντας 3 Hash Tables και Bucket Length 100.0. Με αυτές τις ρυθμίσεις, ο αλγόριθμος πέτυχε εντυπωσιακή ανάκληση της τάξης του 99.81% σε μόλις 1.19 δευτερόλεπτα. Πρόκειται για μια δραματική μείωση του χρόνου σε σχέση με τα 70.01 δευτερόλεπτα της βασικής μεθόδου, χάνοντας ελάχιστα, σχεδόν μηδαμινά, πραγματικά ζευγάρια.

#### 4.4 Κλιμακωσιμότητα σε Μεγάλα Δεδομένα (SIFT1M)

Το τελικό και πιο απαιτητικό στάδιο της αξιολόγησης αφορούσε την εφαρμογή των βέλτιστων παραμέτρων που βρέθηκαν παραπάνω στο πλήρες, μεγάλο σύνολο δεδομένων (SIFT1M). Ολόκληρη η βάση των 1.000.000 διανυσμάτων φορτώθηκε επιτυχώς στην κύρια μνήμη (RAM) του συστήματος. Ωστόσο, αναδείχθηκε ένας σημαντικός φυσικός περιορισμός υλικού (hardware limit): η ταυτόχρονη επεξεργασία 10.000 ερωτημάτων (queries) οδηγούσε το δωρεάν περιβάλλον εκτέλεσης (με όριο τα ~12GB RAM) σε OOM (Out Of Memory – εξάντληση διαθέσιμης μνήμης).

Για να καταστεί εφικτή η εκτέλεση και να αποδειχθεί στην πράξη η κλιμακωσιμότητα (scalability) του κώδικα, το τελικό πείραμα διεξήχθη αντιπαραβάλλοντας ένα αντιπροσωπευτικό δείγμα 100 ερωτημάτων (queries) έναντι ολόκληρης της βάσης του ενός εκατομμυρίου

διανυσμάτων, με το κατώφλι απόστασης να παραμένει στο  $\theta = 250.0$ .

Συγκρίθηκαν δύο διαφορετικές προσεγγίσεις:

Εκτέλεση. Βάση: 1000000 | Queries: 100

ΤΕΛΙΚΗ ΣΥΓΚΡΙΣΗ (Στο SIFT1M για  $\theta=250.0$  και 100 queries)  
Μοντέλο 1 (Ταχύτητα): Βρήκε 125416 ζευγάρια σε 26.08 sec  
Μοντέλο 2 (Ακρίβεια): Βρήκε 128443 ζευγάρια σε 59.66 sec

- Μοντέλο 1 - Ταχύτητα (1 Hash Table, Bucket Length 300.0): Το σύστημα έδωσε προτεραιότητα στον γρήγορο χρόνο απόκρισης. Ολοκλήρωσε την αναζήτηση ταχύτατα σε 26.08 δευτερόλεπτα, εντοπίζοντας 125.416 ζευγάρια.
- Μοντέλο 2 - Ακρίβεια (3 Hash Tables, Bucket Length 100.0): Το σύστημα χρησιμοποίησε τις βέλτιστες παραμέτρους του Grid Search δίνοντας προτεραιότητα στην ορθότητα. Χρειάστηκε υπερδιπλάσιο χρόνο (59.66 δευτερόλεπτα), αλλά κατάφερε να εντοπίσει 128.443 ζευγάρια (προσφέροντας περίπου 3.000 περισσότερα σωστά αποτελέσματα).

Το πείραμα αυτό επιβεβαιώνει απόλυτα την αναμενόμενη θεωρητική συμπεριφορά του αλγορίθμου σε συνθήκες Μεγάλων Δεδομένων: η προσθήκη περισσότερων πινάκων κατακερματισμού (Hash Tables) αυξάνει τον υπολογιστικό φόρτο και τον χρόνο εκτέλεσης, αλλά αποτελεί εγγύηση για σημαντικά υψηλότερη ακρίβεια στα τελικά αποτελέσματα.

#### 4.5 Εκτέλεση σε Κατανεμημένο Περιβάλλον (Hadoop YARN Cluster)

Με στόχο την αξιολόγηση της αρχιτεκτονικής σε ρεαλιστικές συνθήκες Μεγάλων Δεδομένων, το σύστημα αναπτύχθηκε επιπλέον σε ένα πραγματικό, πολυ-κομβικό κατανεμημένο περιβάλλον (Cluster). Για τον σκοπό αυτό, επιστρατεύτηκαν Εικονικές Μηχανές (VMs) από την εθνική υποδομή ~Okeanos-GRNET. Στήθηκε μια συστοιχία με διαχειριστή πόρων το Hadoop YARN, αποτελούμενη από έναν κεντρικό κόμβο (Master) και τρεις κόμβους επεξεργασίας (Worker Nodes). Για την αποθήκευση και διαχείριση των δεδομένων αξιοποιήθηκε το Κατανεμημένο Σύστημα Αρχείων HDFS, ενώ η υποβολή της εργασίας πραγματοποιήθηκε μέσω του `spark-submit`. Δύο κρίσιμες παρατηρήσεις προέκυψαν από αυτή τη δοκιμή: Πρώτον, η βασική μέθοδος (Cross Join)

ολοκληρώθηκε σε 193 δευτερόλεπτα. Η αύξηση αυτού του χρόνου σε σχέση με την τοπική εκτέλεση δικαιολογείται πλήρως από το κόστος επικοινωνίας δικτύου (network overhead) μεταξύ των φυσικών κόμβων του cluster. **Δεύτερον**, η εφαρμογή του προτεινόμενου αλγορίθμου LSH ανέδειξε νέες βέλτιστες υπερπαραμέτρους (3 Hash Tables, 300.0 Bucket Length) και μείωσε δραματικά τον χρόνο σύζευξης στα 7.3 δευτερόλεπτα, επιτυγχάνοντας ταυτόχρονα το εντυπωσιακό ποσοστό ανάκλησης (Recall) 99.94%. Η διαφορά ταχύτητας (από 193s σε 7.3s) αποδεικνύει την τεράστια υπεροχή του LSH σε περιβάλλοντα δικτύου.

```
Ξεκινάει η φόρτωση των δεδομένων...
Μετατροπή και αποθήκευση στο HDFS (Hadoop)...
Φορτώθηκαν επιτυχώς!
=====
Υπολογισμός Ground Truth (Cross Join)
Ολοκληρώθηκε σε: 193.04 δευτερόλεπτα
Για  $\theta=250.0$ , βρέθηκαν 1613 ζευγάρια.
+-----+-----+-----+
|base_id|query_id|euclidean_dist|
+-----+-----+-----+
| 61|      11|    241.64644|
|118|      11|    246.58467|
| 31|      22|    231.64629|
|105|      22|    208.31706|
|107|      19|    236.90082|
+-----+-----+-----+
only showing top 5 rows

To Ground Truth αποθηκεύτηκε στο HDFS!
=====
Grid Search LSH (15 Συνδυασμοί). Πραγματικά ζευγάρια: 1613
Ολοκληρώθηκε σε 24.96s (Recall: 80.60%) - HT:1, BL:100.0
Ολοκληρώθηκε σε 7.72s (Recall: 80.97%) - HT:1, BL:200.0
Ολοκληρώθηκε σε 6.64s (Recall: 80.97%) - HT:1, BL:300.0
Ολοκληρώθηκε σε 6.41s (Recall: 80.97%) - HT:1, BL:400.0
Ολοκληρώθηκε σε 5.93s (Recall: 80.97%) - HT:1, BL:500.0
Ολοκληρώθηκε σε 7.52s (Recall: 98.33%) - HT:2, BL:100.0
Ολοκληρώθηκε σε 6.93s (Recall: 98.39%) - HT:2, BL:200.0
Ολοκληρώθηκε σε 6.42s (Recall: 98.39%) - HT:2, BL:300.0
Ολοκληρώθηκε σε 7.24s (Recall: 98.39%) - HT:2, BL:400.0
Ολοκληρώθηκε σε 6.87s (Recall: 98.39%) - HT:2, BL:500.0
Ολοκληρώθηκε σε 7.22s (Recall: 99.81%) - HT:3, BL:100.0
Ολοκληρώθηκε σε 7.78s (Recall: 99.94%) - HT:3, BL:200.0
Ολοκληρώθηκε σε 7.40s (Recall: 99.94%) - HT:3, BL:300.0
Ολοκληρώθηκε σε 7.97s (Recall: 99.94%) - HT:3, BL:400.0
Ολοκληρώθηκε σε 8.11s (Recall: 99.94%) - HT:3, BL:500.0
```

| --- ΑΠΟΤΕΛΕΣΜΑΤΑ GRID SEARCH ---                      |               |              |          |       |            |  |
|-------------------------------------------------------|---------------|--------------|----------|-------|------------|--|
| Hash Tables                                           | Bucket Length | Χρόνος (sec) | Βρέθηκαν | Σωστά | Recall (%) |  |
| 1                                                     | 100.0         | 24.96        | 1300     | 1300  | 80.60      |  |
| 1                                                     | 200.0         | 7.72         | 1306     | 1306  | 80.97      |  |
| 1                                                     | 300.0         | 6.64         | 1306     | 1306  | 80.97      |  |
| 1                                                     | 400.0         | 6.41         | 1306     | 1306  | 80.97      |  |
| 1                                                     | 500.0         | 5.93         | 1306     | 1306  | 80.97      |  |
| 2                                                     | 100.0         | 7.52         | 1586     | 1586  | 98.33      |  |
| 2                                                     | 200.0         | 6.93         | 1587     | 1587  | 98.39      |  |
| 2                                                     | 300.0         | 6.42         | 1587     | 1587  | 98.39      |  |
| 2                                                     | 400.0         | 7.24         | 1587     | 1587  | 98.39      |  |
| 2                                                     | 500.0         | 6.87         | 1587     | 1587  | 98.39      |  |
| 3                                                     | 100.0         | 7.22         | 1610     | 1610  | 99.81      |  |
| 3                                                     | 200.0         | 7.78         | 1612     | 1612  | 99.94      |  |
| 3                                                     | 300.0         | 7.40         | 1612     | 1612  | 99.94      |  |
| 3                                                     | 400.0         | 7.97         | 1612     | 1612  | 99.94      |  |
| 3                                                     | 500.0         | 8.11         | 1612     | 1612  | 99.94      |  |
| ΝΙΚΗΤΗΣ (Βέλτιστο Trade-off):                         |               |              |          |       |            |  |
| ♦ Hash Tables:                                        | 3             |              |          |       |            |  |
| ♦ Bucket Length:                                      | 300.0         |              |          |       |            |  |
| ♦ Recall:                                             | 99.94%        |              |          |       |            |  |
| ♦ Χρόνος:                                             | 7.4 sec       |              |          |       |            |  |
| =====                                                 |               |              |          |       |            |  |
| ΤΕΛΙΚΗ ΣΥΓΚΡΙΣΗ (Ταχύτητα vs Ακρίβεια)                |               |              |          |       |            |  |
| Μοντέλο 1 (Ταχύτητα): Βρήκε 1306 ζευγάρια σε 5.21 sec |               |              |          |       |            |  |
| Μοντέλο 2 (Ακρίβεια): Βρήκε 1612 ζευγάρια σε 7.30 sec |               |              |          |       |            |  |

## 5. Συμπεράσματα

Η παρούσα εργασία επικεντρώθηκε στην επίλυση του προβλήματος της ταχείας αναζήτησης όμοιων πολυδιάστατων διανυσμάτων σε συνθήκες Μεγάλων Δεδομένων. Μέσα από την υλοποίηση και την πειραματική αξιολόγηση, εξάγονται τα ακόλουθα βασικά συμπεράσματα:

Πρώτον, επιβεβαιώθηκε στην πράξη ότι η παραδοσιακή μέθοδος εξαντλητικής αναζήτησης είναι πρακτικά ανεφάρμοστη σε κλίμακα εκατομμυρίων εγγραφών. Το τεράστιο υπολογιστικό κόστος που απαιτείται για τον υπολογισμό του καρτεσιανού γινομένου καθιστά αναγκαία τη μετάβαση σε προσεγγιστικούς αλγορίθμους.

Δεύτερον, αποδείχθηκε ότι ο αλγόριθμος Locality-Sensitive Hashing (LSH), και συγκεκριμένα η τεχνική Bucketed Random Projection, αποτελεί μια εξαιρετικά αποδοτική λύση. Η ικανότητα του αλγορίθμου να ομαδοποιεί "έξυπνα" τα κοντινά διανύσματα επιτρέπει στο σύστημα να αποφεύγει εκατομμύρια άσκοπες συγκρίσεις.

Τρίτον, η επιλογή των σωστών υπερπαραμέτρων αποτελεί τον πιο κρίσιμο παράγοντα επιτυχίας του LSH. Όπως έδειξε η διαδικασία του Grid Search, δεν υπάρχει μια απόλυτα "σωστή" ρύθμιση, αλλά μια χρυσή τομή (trade-off) μεταξύ ταχύτητας και ακρίβειας. Για το συγκεκριμένο σύνολο δεδομένων, η χρήση 3 πινάκων κατακερματισμού (Hash Tables) και μήκους κάδου (Bucket Length) 100.0, αποδείχθηκε η ιδανική επιλογή, προσφέροντας σχεδόν απόλυτη ανάκληση (99.81%) σε ελάχιστο χρόνο (1.19 δευτερόλεπτα στο μικρό δείγμα). Η προσθήκη επιπλέον

πινάκων κατακερματισμού, όπως επαληθεύτηκε και στο μεγάλο σύνολο SIFT1M, αυξάνει την ορθότητα των αποτελεσμάτων, με τίμημα τον διπλασιασμό του χρόνου εκτέλεσης.

Τέλος, η ανάπτυξη του συστήματος στο περιβάλλον του Apache Spark αποτέλεσε τον ακρογωνιαίο λίθο για την κλιμακωσιμότητα της λύσης. Παρά τους αυστηρούς περιορισμούς μνήμης του διαθέσιμου υλικού (Google Colab, 12GB RAM), η εφαρμογή τεχνικών διαχείρισης μνήμης (caching), σωστής κατάτμησης (partitioning) και παράλληλης επεξεργασίας σε πολλαπλούς πυρήνες, επέτρεψε την επιτυχή φόρτωση και επεξεργασία του συνόλου 1.000.000 διανυσμάτων, αποδεικνύοντας ότι ο κώδικας είναι έτοιμος να εκτελεστεί απρόσκοπτα σε μεγαλύτερα, βιομηχανικά clusters.

## ΒΙΒΛΙΟΓΡΑΦΙΑ

1. Apache Spark Documentation. "Locality-Sensitive Hashing (LSH) - Spark MLlib". Διαθέσιμο στο: <https://spark.apache.org/docs/latest/ml-features.html#locality-sensitive-hashing>
2. Jégou, H., Douze, M., & Schmid, C. (2011). "Product Quantization for Nearest Neighbor Search". IEEE Transactions on Pattern Analysis and Machine Intelligence. (Πηγή για το σύνολο δεδομένων SIFT1M: <http://corpus-texmex.irisa.fr/>)
3. Leskovec, J., Rajaraman, A., & Ullman, J. D. (2014). "Mining of Massive Datasets" (Κεφάλαιο 3: Finding Similar Items). Cambridge University Press.